



DEEP LEARNING AND NATURAL LANGUAGE PROCESSING

LECTURE 11: AI AGENTS, AGENTIC WORKFLOWS, AND THE INDUSTRY LANDSCAPE

Dr. Adewale Akinfaderin

SEAS 8520-D6A · Summer 2026



Table of Contents

1 From Models to Systems

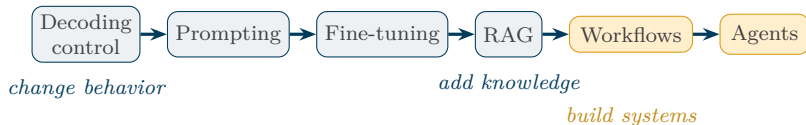
- ▶ From Models to Systems
- ▶ Workflows versus Agents
- ▶ Five Workflow Patterns
- ▶ Anatomy of an Agent
- ▶ Multi-Agent Systems
- ▶ Protocols
- ▶ Evaluating Agents
- ▶ The Industry Landscape
- ▶ Wrap-Up
- ▶ References

Where we have been. Earlier lectures changed the model's *behavior* (decoding control, prompting, LoRA and QLoRA fine-tuning), then gave the model *knowledge* it never trained on (retrieval, embeddings, vector databases, the RAG pipeline). The model was the unit of design throughout.

Today. We change the unit of design from the model to the **system**. Once a model calls tools, decides its own next step, and coordinates with other models, prompting has become system design, and it needs its own vocabulary.

1. **Workflows versus agents:** who controls the flow, code or the model.
2. **Five workflow patterns:** chaining, routing, parallelization, orchestrator-workers, evaluator-optimizer.
3. **Anatomy of an agent:** the loop, tool calling, structured output, memory.
4. **Multi-agent systems:** orchestration, termination, when one agent is enough.
5. **Protocols:** MCP for tools, A2A for agents.
6. **Evaluating agents:** trajectories, judges, failure modes, guardrails.
7. **Industry landscape:** serving, quantized inference, cost, deployment.

Every technique in this course sits on one axis: how much of the work the *system around the model* does versus the model itself.



The move today. Left of RAG, a human writes every control decision. At the far right, the model writes them. Everything between is a question of how much control you hand over, and today's job is to decide that deliberately rather than by default.



Table of Contents

2 Workflows versus Agents

- ▶ From Models to Systems
- ▶ Workflows versus Agents
- ▶ Five Workflow Patterns
- ▶ Anatomy of an Agent
- ▶ Multi-Agent Systems
- ▶ Protocols
- ▶ Evaluating Agents
- ▶ The Industry Landscape
- ▶ Wrap-Up
- ▶ References

Both systems below are “agentic.” The difference that organizes this whole lecture is who holds control over the process.

Workflows

LLMs and tools orchestrated through **predefined code paths**. A human writes the control flow in advance. The model fills in each step but does not choose the sequence.

Predictable and consistent.

Agents

LLMs **dynamically direct their own processes** and tool usage, keeping control over how they accomplish a task. The model decides which steps to take and in what order.

Flexible and model-driven.

Why the line matters. A workflow is a program with model-shaped holes; you can read its control flow off the source code. An agent’s control flow exists only at runtime, produced by the model. That single difference drives everything downstream: how you debug it, how much it costs, and how you evaluate it.

The most important advice in the agents literature is a warning against building agents.

The rule

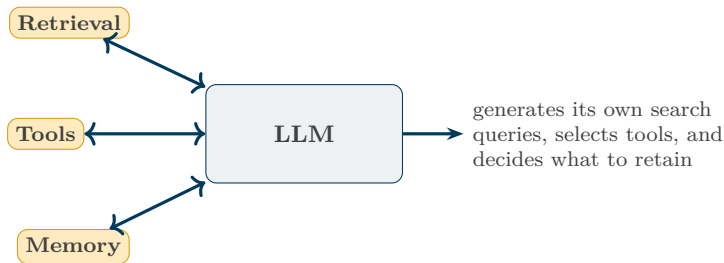
Find the simplest solution possible, and increase complexity only when it demonstrably improves outcomes. For many applications, optimizing a single LLM call with retrieval and in-context examples is enough.

The tradeoff you are buying into. Agentic systems trade **latency and cost** for better task performance. A single call answers in one round trip. An agent may take twenty rounds, each a full model call, before it stops.

A decision rule.

- Well-defined task, known steps → **workflow** for predictability.
- Open-ended task, steps unknowable in advance → **agent** for flexibility.
- Neither clearly applies → a single augmented call, and measure before adding more.

Every pattern in this lecture is composed from one unit: a model given three capabilities it can invoke on its own.



Retrieval is last lecture's RAG, now something the model triggers rather than a fixed pipeline step. **Tools** are functions the model can call. **Memory** is what it carries across steps. Two design rules follow: tailor the capabilities to your use case, and give the model an easy, well-documented interface. Hold that second rule; it returns as tool design.



Table of Contents

3 Five Workflow Patterns

- ▶ From Models to Systems
- ▶ Workflows versus Agents
- ▶ **Five Workflow Patterns**
- ▶ Anatomy of an Agent
- ▶ Multi-Agent Systems
- ▶ Protocols
- ▶ Evaluating Agents
- ▶ The Industry Landscape
- ▶ Wrap-Up
- ▶ References

Before agents give up control entirely, five workflow patterns cover most production needs. Each composes augmented LLMs through code you write. The art is matching the pattern to the task.

Pattern	Use when
Prompt chaining	the task splits into fixed sequential steps
Routing	inputs fall into distinct categories handled separately
Parallelization	subtasks are independent, or you want multiple attempts
Orchestrator-workers	subtasks cannot be predicted in advance
Evaluator-optimizer	clear criteria exist and iteration measurably helps

Reading the list. The first three have control flow fixed at design time. Orchestrator-workers pushes one decision (what the subtasks are) to runtime. Evaluator-optimizer adds a loop. Complexity rises as you read down.

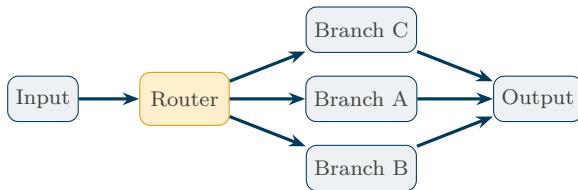
Definition. Decompose a task into a sequence of steps, where each LLM call processes the output of the previous one. Programmatic checks, called gates, can sit between steps and stop the chain when a step fails.



When to use it. The task decomposes cleanly into fixed subtasks, and you are willing to trade latency for accuracy, since each step is a simpler ask than the whole.

Examples. Generate marketing copy, then translate it. Write a document outline, check it against criteria at the gate, then write the full document only if the outline passes.

Definition. Classify an input, then direct it to a specialized follow-up task. One classifier call decides which branch runs.



When to use it. A complex task splits into distinct categories better handled separately, and classification can be done accurately. Separating concerns lets you tune each branch without breaking the others.

Examples. Route customer-service queries to general questions, refunds, or technical support. Route easy questions to a small cheap model and hard ones to a larger model, which cuts cost without lowering quality on the questions that need it.

Definition. Several LLM calls work at the same time and their outputs are aggregated in code. Two variations:

Sectioning

Break a task into **independent subtasks** run in parallel. Each call handles one section; the aggregator stitches them.

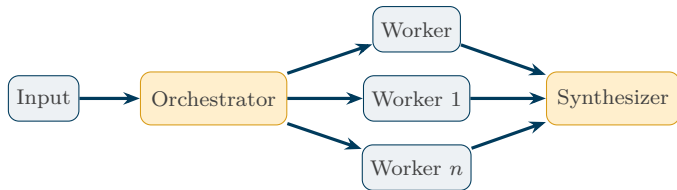
When to use it. Subtasks parallelize for speed (sectioning), or multiple attempts raise confidence (voting).

Examples. *Sectioning:* one model screens a query for safety while another generates the response, a guardrail that keeps the two concerns separate. *Voting:* several prompts review code for vulnerabilities, and an issue flagged past a vote threshold is escalated.

Voting

Run the **same task** several times for diverse outputs, then combine by majority or threshold.

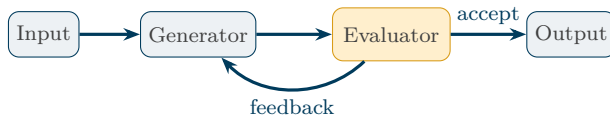
Definition. A central LLM dynamically breaks a task into subtasks, delegates each to a worker LLM, and synthesizes their results.



The key contrast with parallelization. Here the subtasks are **not predefined**. The orchestrator determines them from the specific input, so you cannot draw the branches in advance.

Examples. A coding tool that changes an unpredictable set of files each task. A search task that gathers and combines information from sources chosen on the fly.

Definition. One LLM call generates a response while another evaluates it and returns feedback, in a loop, until the evaluator accepts.



When to use it. Two conditions together: clear evaluation criteria exist, and responses measurably improve when feedback is articulated. A good test is whether a human giving written feedback would help; if so, an evaluator LLM can often stand in.

Examples. Literary translation, where the evaluator catches nuance the generator then refines. Multi-round research, where each round is evaluated for what is still missing.



Table of Contents

4 Anatomy of an Agent

- ▶ From Models to Systems
- ▶ Workflows versus Agents
- ▶ Five Workflow Patterns
- ▶ **Anatomy of an Agent**
- ▶ Multi-Agent Systems
- ▶ Protocols
- ▶ Evaluating Agents
- ▶ The Industry Landscape
- ▶ Wrap-Up
- ▶ References

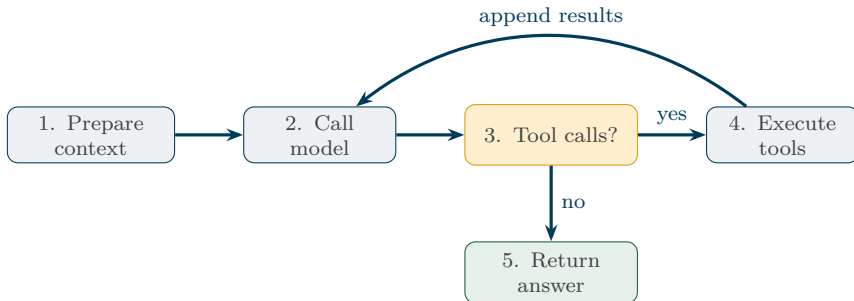
The five patterns fixed the control flow in code. An agent removes that scaffolding: the model decides what to do next based on what it observes. That handover of control is the dividing line, and the mechanics below make it concrete.

An agent, concretely

A model equipped with three capabilities: **reasoning** (the model itself), **acting** (tools it can call), and **adaptation** (memory it can read and write). A single “run” spans many model calls and tool calls until a stopping condition fires.

The shift in what you write. A workflow is code that calls a model. An agent is a model that calls your code. You still write the tools, the stopping conditions, and the guardrails, and the model decides when to use each.

Every agent run is one loop: a five-step cycle that repeats until the model answers without asking for another tool.



Context is the task, instructions, memory, and message history. The model either emits text (done) or emits tool calls. Tool results are appended to the history, and the loop returns to the model. A `max_iterations` guard (commonly 10) stops runaway loops.

A tool is a function the model can invoke. The framework advertises each function to the model as a JSON schema of its name and parameters; the model emits a structured call; your code runs it and appends the result.

```
1 def get_weather(location: str):  
2     """Current weather for a city.  
3     """  
4     ...  
5     return {"tempC": 14}
```

A plain function becomes a tool.

```
1 # advertised to the model  
2 {"name": "get_weather",  
3  "parameters": {"location":  
4                  {"type": "string"}}}  
5 # model emits  
6 get_weather(location="Paris")
```

The model chooses to call it.

The round trip. Model emits `get_weather(location="Paris")` → framework runs it → result `{"tempC": 14}` appended as a tool message → model reads it and writes a natural-language answer. The model never runs code; it only requests calls.

Free text is unpredictable to parse. Constraining a model's output to a fixed schema turns it into data your code can act on, and this is the single most important lever for reliability.

Without structure

“I think we should probably route this to the refunds team, since...”

Your code must parse intent from prose.

With a schema

```
{"route": "refunds",  
 "confidence": 0.9}
```

Your code reads a field.

Why it matters downstream. Routing, plan generation, and picking the next agent all depend on the model returning a value your program can branch on. Structured output is what makes the orchestrator and router patterns from earlier actually implementable.

Memory is what an agent carries across steps. It splits two ways, and the second closes the loop to last lecture.

Short-term

The message history *within a single run*. Cheap, automatic, bounded by the context window. Gone when the run ends.

Long-term

Retrieval over a vector store (Chroma, Qdrant, Pinecone) *across runs*. This is RAG serving as memory.

Two ways to manage long-term memory. *Application-managed*: your code decides what to store and injects it into context. *Agent-managed*: the model treats memory as a tool it reads and writes itself, which enables cross-session learning. The tradeoff is real: memory adds retrieval latency and storage cost, so a stateless agent is simpler and faster when each task stands alone.



Table of Contents

5 Multi-Agent Systems

- ▶ From Models to Systems
- ▶ Workflows versus Agents
- ▶ Five Workflow Patterns
- ▶ Anatomy of an Agent
- ▶ **Multi-Agent Systems**
- ▶ Protocols
- ▶ Evaluating Agents
- ▶ The Industry Landscape
- ▶ Wrap-Up
- ▶ References

The instinct to split a task across many specialized agents is often wrong. A well-designed single agent usually beats a poorly-designed multi-agent system, and most tasks never need more than one.

Reach for multiple agents only when the task has

- **Decomposability** into distinct steps with clear boundaries.
- **Diverse expertise** that benefits from different roles or perspectives.
- **Extensive context** per step that would overload one agent's window.
- **Adaptive requirements**, where the solution path is unknown until you act.

The default. Translate a document, summarize a report, answer one question: a single agent handles all of these. Multi-agent design earns its coordination cost only when at least two of the conditions above hold.

A multi-agent system needs a rule for choosing which agent acts next. Orchestration is one loop with three interchangeable selection strategies.

Strategy	How the next agent is chosen	Cost
Round-robin	fixed cyclic order	cheapest, no extra call
AI-driven	an LLM reads the conversation and picks	one call per turn
Plan-based	an LLM writes a plan, then executes and monitors its steps	planning call plus monitoring

Reading the ladder. Round-robin is predictable and needs no reasoning, which makes it a strong prototype. AI-driven selection adapts turn by turn at the cost of a model call each time. Plan-based orchestration adds metacognition: the system commits to a plan, checks progress against it, and retries failed steps.

An agent loop with no stopping condition is a way to spend money without producing an answer. Treat termination as a first-class, composable object.

Common termination conditions

- **Max messages:** hard cap on turns.
- **Text mention:** stop when an agent emits a sentinel like `DONE`.
- **Token usage:** stop when a budget is exhausted.
- **Timeout:** stop after a wall-clock limit.

Compose them. Conditions combine with boolean logic, so a production system rarely relies on one:

```
MaxMessages(20) | TextMention("DONE") | Timeout(300s)
```

The run ends when any of the three fires, which bounds cost, honors an explicit finish signal, and protects against a hang all at once.

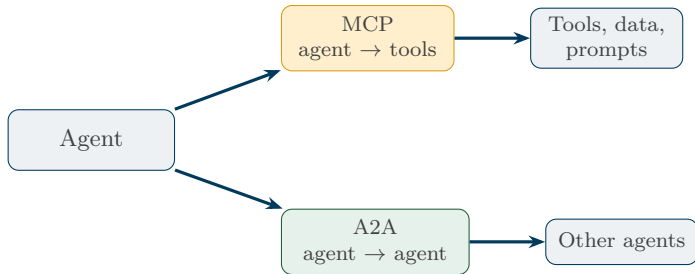


Table of Contents

6 Protocols

- ▶ From Models to Systems
- ▶ Workflows versus Agents
- ▶ Five Workflow Patterns
- ▶ Anatomy of an Agent
- ▶ Multi-Agent Systems
- ▶ **Protocols**
- ▶ Evaluating Agents
- ▶ The Industry Landscape
- ▶ Wrap-Up
- ▶ References

Once agents call external tools and talk to other agents, you hit a combinatorial mess. Every tool needs a custom integration for every agent, and every agent partnership needs its own glue. Two open protocols standardize the two boundaries.



The payoff of standardizing. Write one MCP server and every MCP-aware host can use it. Publish one agent card and any A2A client can delegate work to you. Standard interfaces replace $N \times M$ custom integrations with $N + M$.

The Model Context Protocol (MCP), released in late 2024, standardizes how an agent connects to tools and data. Its slogan: write one server, use it everywhere.

Three layers.

- **Server:** exposes *tools* (functions), *resources* (readable data by URI), and *prompts* (reusable templates).
- **Client:** a one-to-one connection that speaks the protocol.
- **Host:** the user-facing app embedding clients (Claude Desktop, Cursor, VS Code).

Transports

stdio for a local subprocess;
streamable HTTP for a remote server.

Why it is more than function calling. MCP servers can pause to request structured input from the user (elicitation), ask the host's model for a completion while the host keeps the API keys (sampling), and resume interrupted streams. These let a tool behave like a small agent while the host stays in control.

Where MCP connects an agent to tools, the Agent-to-Agent protocol (A2A), released in 2025 and now under the Linux Foundation, connects an agent to *other agents* it treats as opaque black boxes. It is task-centric: you delegate a task and track its state.

Core elements

- **Agent Card:** JSON at a well-known URL declaring identity, endpoint, skills, and auth.
- **Task:** stateful, uniquely identified, immutable once terminal.
- **Artifact:** a concrete output, streamable.

Task lifecycle

Active: submitted, working.

Interrupted: input-required, auth-required.

Terminal: completed, failed, canceled, rejected.

The mental model. You discover an agent by reading its card, send it a task over HTTP, and poll or stream until the task reaches a terminal state. You never see how it works inside.

The two protocols are complementary, sitting at different layers of the same system.

	MCP	A2A
Connects	agent to tools, data, prompts	agent to other agents
Unit	individual tool call	delegated task
You get	visibility and control over each call	autonomy behind a black box
Use for	composing fine-grained capabilities	handing off whole tasks across organizations

Use both. A production agent might expose its own skills through an agent card (A2A) while reaching for a database and a filesystem through MCP servers. The choice is about the layer, and a real system standardizes both.



Table of Contents

7 Evaluating Agents

- ▶ From Models to Systems
- ▶ Workflows versus Agents
- ▶ Five Workflow Patterns
- ▶ Anatomy of an Agent
- ▶ Multi-Agent Systems
- ▶ Protocols
- ▶ **Evaluating Agents**
- ▶ The Industry Landscape
- ▶ Wrap-Up
- ▶ References

A single model call has one output to grade. An agent produces a **trajectory**: a sequence of reasoning messages and tool actions leading to a final answer. The core insight is that whether you evaluate one model, one agent, or a whole team, you evaluate the same object.

Evaluation-driven development

Define success criteria *before* building, the way test-driven development defines tests first: state the criteria, build a task suite, choose metrics, establish a baseline, then iterate.

The answer-extraction problem. With a tool-using agent, where is the answer? The last message is often “The calculation is complete,” with the actual number two turns back. Naive last-message grading fails, so judges need an explicit strategy for which message holds the answer.

Reference-based

Compare against a known answer: exact match, contains, fuzzy match. Use when ground truth exists and reproducibility matters. Cheap and deterministic.

Reference-free

An LLM judge scores dimensions like helpfulness or efficiency, 0 to 10 with reasoning. Use for nuanced quality with no single right answer.

Grade the process, not only the answer. An agent can reach the right answer down a wrong reasoning path and get lucky. Process metrics inspect the tool messages: did it select the right tools in the right order? This catches trajectories that pass on output but would fail on a slightly different input.

Judges have biases too. A documented failure is a verbosity-penalty bug, where the judge marks concise correct answers down. Calibrate the judge before trusting it, the same lesson as last lecture's self-preference bias.

More autonomy is not free, and it is often not better. Consider four configurations run on the same tasks, from a direct model call to a fully AI-orchestrated multi-agent team.

Task type	Best config	Token cost
Simple reasoning	direct model (9.7/10)	1×
Tool-heavy	multi-agent (9.2 vs 6.8)	up to 43×

Reading the result. On simple reasoning, the multi-agent team scored no higher than a single direct call while costing far more. On tool-heavy work, agents clearly won because the task genuinely needed tools. The lesson is the start-simple rule again, now with a number attached: autonomy has unbounded cost, so add it only where a measurement shows it pays.

Ten recurring ways agent systems fail in production share one theme: the system was given autonomy without the scaffolding autonomy needs.

- **Vague instructions.** Good agent prompts can run pages long.
- **Too many tools.** Ten excellent tools beat fifty mediocre ones; favor reliability over breadth.
- **No termination logic.** Runaway loops or premature exits.
- **No evaluation.** You cannot optimize what you do not measure.
- **Wrong pattern.** Multi-agent where one agent would do.
- **No memory or reflection.** The agent cannot learn or self-correct.
- **Never delegating to a human.** No escape hatch when the agent is stuck.

The through-line. Each failure is a missing control: an instruction, a stopping rule, a metric, a human. Agentic autonomy is only as safe as the scaffolding around it.

The last defense is deciding which actions an agent may take on its own and which need a human to approve. Tie the decision to risk.

Approval	Action type	Example
Never (auto)	read-only, reversible, low-cost	search, read a file
Always (gate)	irreversible, costly, side-effecting	send email, delete record, pay

Calibrate the threshold with evaluation. If more than about 95% of gated actions get approved anyway, the gate is too cautious and is just adding friction. If harmful actions slip through, it is too loose. The right cut sits where the human is catching real mistakes and little else, and only measurement tells you where that is.



Table of Contents

8 The Industry Landscape

- ▶ From Models to Systems
- ▶ Workflows versus Agents
- ▶ Five Workflow Patterns
- ▶ Anatomy of an Agent
- ▶ Multi-Agent Systems
- ▶ Protocols
- ▶ Evaluating Agents
- ▶ The Industry Landscape
- ▶ Wrap-Up
- ▶ References

Agents are what industry increasingly deploys, and deploying them surfaces a set of engineering realities the research framing hides. The rest of the lecture connects the patterns you just learned to how models actually run in production.

The three questions production forces

- **Serving:** how do you run a model fast enough for many users at once?
- **Cost:** what makes a token expensive, and how do you spend fewer?
- **Operations:** how do you know the system is working when a wrong answer still returns 200 OK?

The connecting thread. An agent that takes twenty model calls per task multiplies every one of these concerns twentyfold. The economics of agents are the economics of serving, so understanding serving is understanding whether an agent is affordable.

Text generation splits into two phases with opposite hardware profiles, and every serving decision follows from this asymmetry.

Prefill

Process all prompt tokens in one parallel pass, fill the KV cache, emit the first token.

Compute-bound: the cores are the limit.

Sets *time to first token*.

Decode

Emit one token per pass, reloading all weights and the whole cache each step for tiny arithmetic.

Memory-bandwidth-bound: the GPU waits on memory. Sets *time per output token*.

The number that matters. On an A100, a 70B model processes 10,000 prompt tokens in roughly the time it takes to generate 100 output tokens: a 100-fold per-token gap, because decode reloads the weights 100 separate times. Output is the expensive part, which is why APIs charge three to five times more for it.

Each token stores one key and one value vector per layer per head. Decode reads that entire cache every single step, so cache size, more than model size, limits how many users you can serve at once.

Sizing a 70B model (grouped-query attention, FP16)

$$\underbrace{2}_{K,V} \times \underbrace{80}_{\text{layers}} \times \underbrace{8}_{\text{KV heads}} \times \underbrace{128}_{\text{head dim}} \times \underbrace{4096}_{\text{tokens}} \times \underbrace{2}_{\text{bytes}} \approx 1.3 \text{ GB per sequence}$$

On an 80 GB GPU, after weights, that leaves room for only tens of concurrent sequences. **PagedAttention** (the idea behind vLLM) fixes this by allocating the cache in small fixed pages on demand, the way an operating system pages memory. It cuts wasted cache from 60–80% down to under 5%, which multiplies concurrency four- to eightfold. **Prefix caching** reuses a shared system prompt computed once, cutting time to first token sharply when prompts overlap.

Serving many users means batching their requests, and how you batch determines throughput.

Static batching

Fix a batch, wait for the longest sequence to finish. Shorter sequences sit idle and padded while one long generation holds the whole batch hostage.

Continuous batching

Re-check after every decode step: evict finished sequences, admit waiting ones immediately. No padding, no idle GPU.

The payoff. Continuous batching (also called iteration-level scheduling) delivers two to five times the throughput of static batching under realistic mixed loads, and every production serving system uses it. It is the single largest free lever on serving economics, since it raises the tokens-per-GPU-dollar without touching the model.

Last time quantization appeared, it was QLoRA shrinking a model for *fine-tuning*. At serving time it does something different: it shrinks the model to fit fewer GPUs and to move fewer bytes per decode step, which is exactly what the memory-bound phase needs.

Precision	FP32	FP16	FP8	INT8	INT4
70B footprint	280 GB	140 GB	70 GB	70 GB	35 GB

The trade. A 70B model quantized to 4-bit fits in about 35 GB and runs three to four times faster, with under 1% accuracy loss. Nearly all deployment uses *post-training* quantization: apply it after training with a small calibration set, in minutes to hours, with no retraining. Quantization-aware training is too expensive at this scale.

The methods split by what they quantize, and the split matters because activations misbehave in ways weights do not.

Weight-only (GPTQ, AWQ)

Quantize weights to 4-bit, keep activations at 16-bit. **GPTQ** uses second-order error information; **AWQ** protects the weight channels that hit large activations. Roughly $4\times$ compression, $3\text{--}4\times$ faster decode, $1\text{--}2\%$ quality loss.

Weight and activation (SmoothQuant)

Quantize both to 8-bit. Activations have outlier channels ten to a hundred times larger than the rest; **SmoothQuant** shifts that difficulty from activations into weights with a per-channel scale, so both quantize cleanly. About $2\times$ speedup, under 0.5% loss.

Choosing. Weight-only when the goal is fitting a large model on fewer GPUs. Weight-and-activation when you want to speed up both phases on hardware with fast integer math. The KV cache itself can also be quantized, which directly helps the memory-bound decode step.

The first deployment decision is whether to call someone else's model or run your own. Five factors frame it.

1. **Data privacy:** an API may violate HIPAA, GDPR, or SOC 2.
2. **Latency:** an API adds a round trip and a queue.
3. **Cost at scale:** per-token pricing is cheaper only at low volume.
4. **Operations:** self-hosting means owning updates and on-call.
5. **Model access:** the hardest reasoning may be API-only.

The common pattern. Prototype on an API, then migrate high-volume or privacy-sensitive workloads to self-hosting once they stabilize. Most systems end up hybrid.

The crossover

Past roughly **one million tokens per day** on one model, a self-hosted quantized open-weight model usually beats the API on cost.

A self-hosted 70B on four A100s costs the same eight to twelve dollars an hour whether it serves ten requests or ten thousand, so the whole game is throughput per GPU-dollar.

Three levers dominate.

- **Caching.** Queries follow a power law, so a cache pays. Exact-match, prefix caching (a shared system prompt computed once), and semantic caching (match by embedding similarity) form a ladder of increasing generality.
- **Routing and cascading.** Send easy queries to a cheap model, hard ones to a capable model. *FrugalGPT* matched GPT-4 quality at up to 98% lower cost; *RouteLLM* reached 95% of GPT-4 quality at 85% lower cost.
- **Token optimization.** Compress prompts, cap output length, and right-size the model to the smallest that meets the quality bar.

The connection to agents. Every one of these levers matters more for an agent, because an agent multiplies the token count of a task by the number of steps in its loop.

LLM systems fail in a way ordinary services do not: the response returns 200 OK, at normal latency, fully formed, and the answer is simply wrong. Standard uptime monitoring sees nothing. Four pillars catch what it misses.

- **Performance:** time to first token, time per output token, p95/p99 latency, throughput, queue depth.
- **Quality:** hallucination rate, faithfulness, toxicity, sampled by an LLM judge or a human.
- **Cost:** tokens per query, cost per user or feature, cache hit rate.
- **User feedback:** explicit ratings, and implicit signals like regeneration and abandonment.

Two operational habits. Log the full input and output text of every model call, since a trajectory you cannot see is a trajectory you cannot debug. And sample a handful of random production pairs by hand every day, because aggregate metrics hide the specific failures that erode trust. Autoscale on queue depth and GPU pressure, since CPU utilization stays near zero and never triggers a scale-up.

Four serving frameworks dominate, and they differ by what they optimize.

Framework	Pick it when
vLLM	general-purpose default; introduced PagedAttention, broad model and hardware support
TensorRT-LLM	you are on NVIDIA and want the lowest latency, and accept model-specific compilation
SGLang	you need structured or constrained generation (JSON, grammar), common in agents
TGI	you live in the Hugging Face stack and want built-in quantization

At the edge. Distilled small models (Gemma, LLaMA 3.2 1B/3B) plus 4-bit quantization bring a capable model under 2 GB, and GGUF with llama.cpp runs it on a laptop CPU or Apple Metal. The common production shape is hybrid routing: private and simple requests run on-device, hard ones go to the cloud, which is the same routing lever one layer down.



Table of Contents

9 Wrap-Up

- ▶ From Models to Systems
- ▶ Workflows versus Agents
- ▶ Five Workflow Patterns
- ▶ Anatomy of an Agent
- ▶ Multi-Agent Systems
- ▶ Protocols
- ▶ Evaluating Agents
- ▶ The Industry Landscape
- ▶ **Wrap-Up**
- ▶ References

The course built one arc: change the model's behavior, give it knowledge, then build a system around it. This lecture completed the arc.

- **Start simple.** A single augmented call beats an agent for most tasks. Add autonomy only where a measurement shows it pays, because on simple work a multi-agent team cost 43 times as much for no gain.
- **Workflows before agents.** Five patterns cover most needs with control flow you can read off the code. Hand control to the model only when the steps cannot be known in advance.
- **Autonomy needs scaffolding.** Tools, termination, memory, evaluation, and human approval are the controls that make an agent safe. Every production failure mode is a missing one.
- **Deployment is the constraint.** Serving, quantization, and cost decide whether an agent is affordable, and an agent multiplies all three by the length of its loop.

You can now place any system you meet on the spectrum this course built, from a single decoded token to a multi-agent system serving thousands of users.

The engineer's questions, in order

1. What is the simplest thing that could work: one call, retrieval, a workflow, an agent?
2. If it must be an agent, what tools, stopping conditions, and guardrails does it need?
3. How will I measure its trajectory, and where does a human stay in the loop?
4. What will it cost to serve, and does the autonomy earn that cost?

Looking ahead. The course-review session steps back to the field as a whole: where these capabilities are heading, what remains unsolved, and how today's patterns become tomorrow's defaults.



Table of Contents

10 References

- ▶ From Models to Systems
- ▶ Workflows versus Agents
- ▶ Five Workflow Patterns
- ▶ Anatomy of an Agent
- ▶ Multi-Agent Systems
- ▶ Protocols
- ▶ Evaluating Agents
- ▶ The Industry Landscape
- ▶ Wrap-Up
- ▶ References

Primary sources.

- Anthropic, *Building Effective Agents*, Anthropic Engineering (2024).
<https://www.anthropic.com/engineering/building-effective-agents>
Workflows versus agents, the five workflow patterns, tool design.
- V. Dibia, *Designing Multi-Agent Systems* (2025).
The agent loop, tools and memory, orchestration, protocols, evaluation, failure modes.

Protocol specifications.

- Model Context Protocol (MCP), <https://modelcontextprotocol.io>
- Agent-to-Agent (A2A) Protocol, Linux Foundation.